HaluRISC: Calibrated, Explainable Hallucination Risk Prediction for LLM Outputs

Course: Machine Learning, Section: E

Submitted By

Saiful Alam Sabbir (0112320105)
Md. Atikur Rahaman (0112310298)

Supervision:
Ohidujjaman Tuhin, PhD
Associate Professor, Dept. of Computer Science and Engineering
United International University.

2026

Department of Computer Science and Engineering
United International University

1 Project Overview

Large language models (LLMs) like ChatGPT are very good at writing text that sounds confident and correct, but they sometimes generate information that is wrong or made up — this is called a hallucination. This project builds a complete system that predicts how likely an LLM answer is to be hallucinated. The system is black-box friendly — it only needs the question, the answer, and (optionally) some context, without ever looking inside the LLM. It is lightweight — it uses simple text features and classical machine learning (XGBoost), so it runs in milliseconds on a normal computer, with no GPU and no API fees. It is explainable — for every prediction it shows *why* the answer looks risky (for example: "the answer contradicts the provided context", or "the answer contains numbers that are not supported"). Finally, it is presentable — a polished, interactive web dashboard lets a judge or supervisor test the system live, type any question and answer, and see the risk score and explanation instantly. The project is judged on two tracks: the quality of the machine learning, and the quality of the final product — the dashboard is designed from day one to be the centerpiece of the demo.

2 Motivation

LLMs are now used in real products — chatbots, Q&A systems, content review tools — where a wrong answer has a real cost, and the most common way to detect hallucinations is to ask the LLM itself to judge its own output, which is slow, expensive, and sometimes wrong. A lightweight model trained on simple text features (word overlap, named entities, contradiction with context, numbers, hedging words) can flag risky answers almost instantly and at almost zero cost. A risk score is also more useful than a simple "yes/no" flag, because real systems can decide what to do with a risky answer — show a warning, ask for human review, or fetch better evidence. And because the final result is a working product with a clean UI rather than just a report, the project is fully demo-able and easy to present.

3 Objectives

Research question: Can a lightweight machine learning model predict hallucination risk in LLM answers using simple text features, while giving calibrated risk scores and clear explanations? To answer this, the project has five objectives: (1) build a feature extraction pipeline (about 20–30 features) that captures how well an answer agrees with the question and context — word overlap, named entities, contradiction signals, numbers, hedging words, and semantic similarity; (2) train and honestly compare classical models — a simple heuristic baseline, Logistic Regression, Random Forest, and XGBoost; (3) calibrate the risk scores so that a score of 0.8 truly means "about 80% risk", making the score trustworthy for real use; (4) explain predictions with SHAP, giving a global view of which features matter most plus detailed case studies of individual predictions; and (5) build a polished web dashboard (React + FastAPI) with strong UI/UX — live prediction, risk gauge, explanation panel, and a demo gallery — designed for presentation and judging.

4 Methodology

The pipeline has seven clear steps:

1. Data preparation — load the HaluEval QA dataset, clean the text, and manually inspect 50 random samples to verify label quality before training.
2. Feature extraction — compute features in seven groups: length/style, lexical overlap, entity overlap, NLI contradiction (using a lightweight pre-trained NLI model), numeric consistency, hedging phrases, and semantic similarity.
3. Model training — 5-fold stratified cross-validation, hyperparameter tuning for XGBoost, and every experiment repeated with 3 random seeds, reported as mean ± standard deviation.
4. Model comparison — heuristic baseline, Logistic Regression, Random Forest, and XGBoost, judged on F1, AUROC, precision, recall, and MCC.
5. Calibration — apply Platt scaling to the final model and measure the gain with Expected Calibration Error (ECE) and Brier score.
6. Explanation — SHAP global feature importance plus 3 local case studies; error analysis on 20 wrong predictions (10 false positives, 10 false negatives).

7. Deployment — FastAPI backend serving the trained model, and a React (Vite) dashboard with a clean, modern UI: risk gauge, feature breakdown, and live input panel.

Statistical rigor: model differences are tested with McNemar's test and bootstrap confidence intervals, so conclusions are not based on luck of the split.

External comparison: results are compared with published HaluEval detection baselines (e.g., SelfCheck-GPT) so a judge can see whether the scores are good or bad.

5 Dataset

We use the HaluEval benchmark (EMNLP 2023) — the standard dataset for hallucination evaluation, widely used and well documented.

- Size: 35,000 samples across 4 task types (QA, dialogue, summarization, general).
- Used here: the QA subset (about 10,000 samples). Each sample has a question, an answer, a context, and a label (hallucinated or not).
- Why QA: it is the largest, cleanest, and most comparable subset, and it matches the demo scenario (question → answer → risk score).

6 Expected Outcomes

1. A journal-style project report (methodology, experiments, results, error analysis, limitations).
2. A trained, calibrated XGBoost model with feature ablation and SHAP explanations.
3. Statistical significance tests (McNemar, bootstrap confidence intervals).
4. A polished, judge-ready web dashboard — live risk prediction with clean UI/UX.
5. Reproducible code: dataset preparation, feature extraction, training, and API.

7 Project Plan

The project follows a straightforward pipeline: (1) prepare the HaluEval QA data and manually audit a sample of the labels, (2) extract about 20–30 text features in seven groups, (3) train and compare simple models, calibrate the risk scores, and evaluate them with statistical tests, (4) explain the predictions with SHAP, and (5) wrap the final model in a FastAPI backend and a React dashboard for live demonstration. The first half of the project (Weeks 1–4) focuses on the experiments — data, features, models, and calibration — while the second half (Weeks 5–8) focuses on explanations, the web product, and the final report.

References

1. Li, J., Cheng, X., Zhao, W.X., Nie, J.-Y., Wen, J.-R. (2023). *HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models.* EMNLP 2023.

2. Manakul, P., Liusie, A., Gales, M. (2023). *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models.* EMNLP 2023.

3. Lin, S., Hilton, J., Evans, O. (2022). *TruthfulQA: Measuring How Models Mimic Human Falsehoods.* ACL 2022.

4. Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P.W., Iyyer, M., Zettlemoyer, L., Hajishirzi, H. (2023). *FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation.* EMNLP 2023.

5. Valentin, S., Fu, J., Detommaso, G., Xu, S., Zappella, G., Wang, B. (2024). *Cost-Effective Hallucination Detection for LLMs.* arXiv:2407.21424.

6. Sundaragiri, D., Reddy, L.M., Gunavardhan, P., Navahith, B. (2026). *Framework for Hallucination Detection in Large Language Models.* IJERT, Vol. 15, Issue 04. DOI: 10.5281/zenodo.20025987.

7. Yadav, S., Verma, N.K. (2026). *A Hybrid Framework for Hallucination Detection in Large Language Models.* IEEE Transactions on Artificial Intelligence. DOI: 10.1109/TAI.2026.3653354.

8. Haq, I., Saqib, M., Zhang, Y., Khan, I.A. (2026). *Quantifying Factual Divergence in Generative Models: SHAP-LIME Based Hallucination Score for LLMs.* Multimedia Systems, Vol. 32, Art. 146. DOI: 10.1007/s00530-025-02150-4.